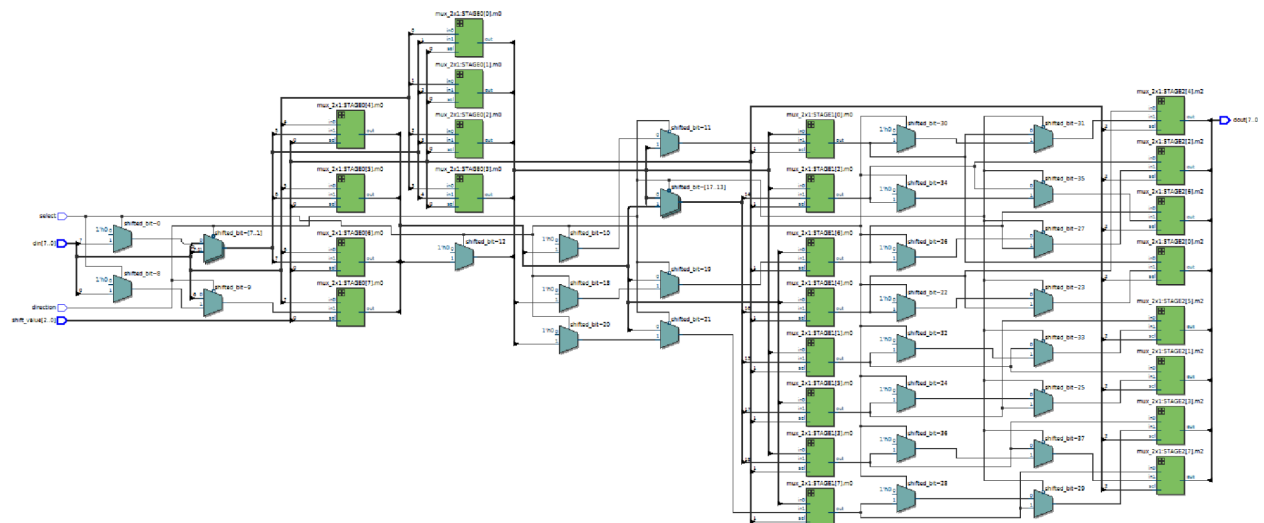
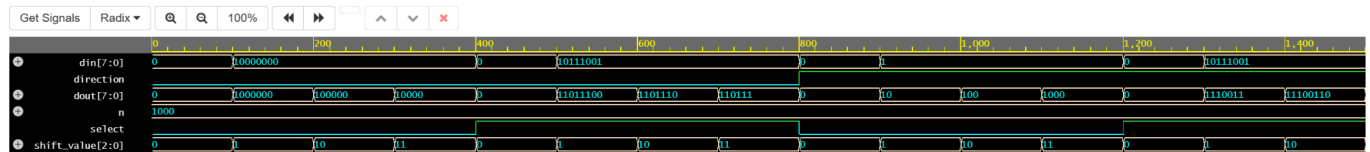




<<Filter>>

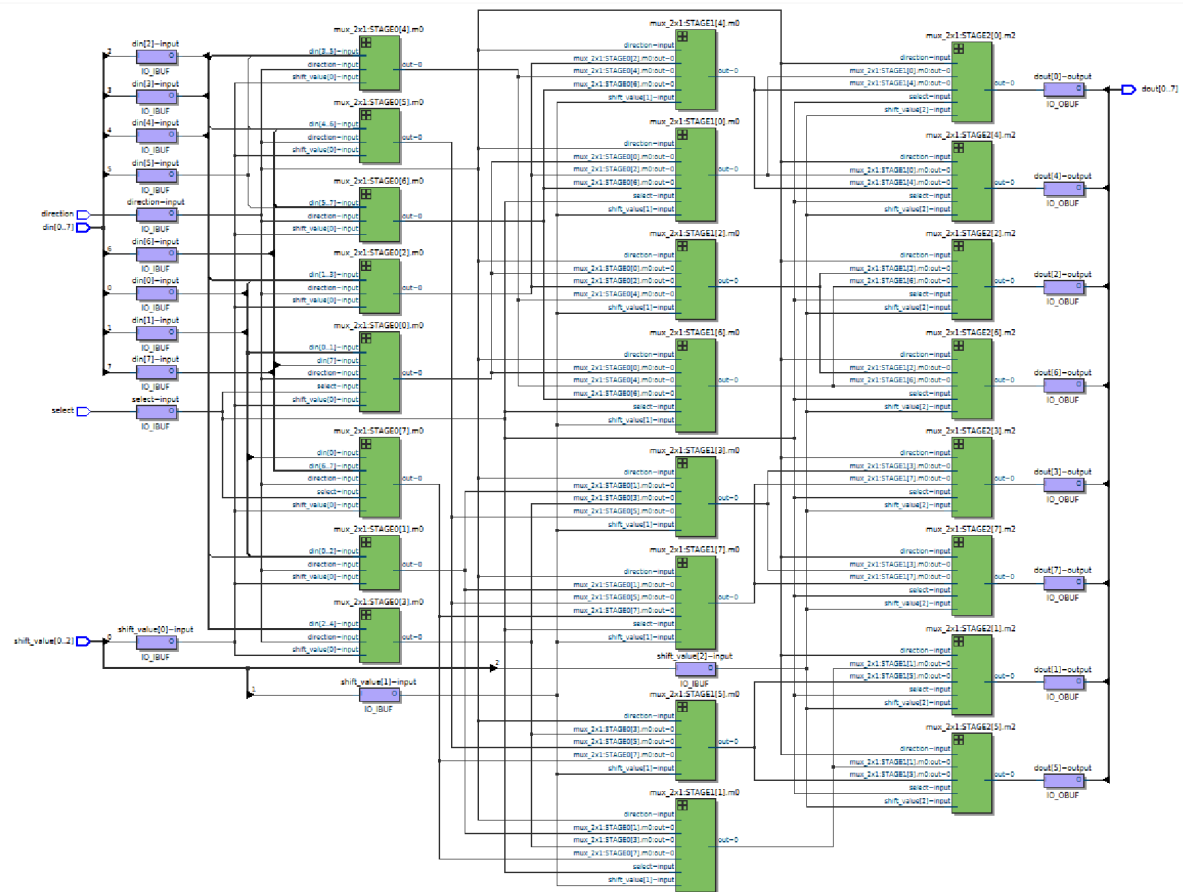
	Resource	Usage
	Estimate of Logic utilization (ALMs needed)	15
▼	Combinational ALUT usage for logic	24
	-- 7 input functions	0
	-- 6 input functions	6
	-- 5 input functions	18
	-- 4 input functions	0
	-- <=3 input functions	0
	Dedicated logic registers	0
	I/O pins	21
	Total DSP Blocks	0
0		
1	Maximum fan-out node	dir...put
2	Maximum fan-out	24
3	Total fan-out	155
4	Average fan-out	2.35



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

```
# run -all
# time=0, select=0 shift_value=000 din=00000000 dout=00000000
# time=100, select=0 shift_value=001 din=10000000 dout=01000000
# time=200, select=0 shift_value=010 din=10000000 dout=00100000
# time=300, select=0 shift_value=011 din=10000000 dout=00010000
# time=400, select=1 shift_value=000 din=00000000 dout=00000000
# time=500, select=1 shift_value=001 din=10111001 dout=11011100
# time=600, select=1 shift_value=010 din=10111001 dout=01101110
# time=700, select=1 shift_value=011 din=10111001 dout=00110111
# time=800, select=0 shift_value=000 din=00000000 dout=00000000
# time=900, select=0 shift_value=001 din=00000001 dout=00000010
# time=1000, select=0 shift_value=010 din=00000001 dout=00000100
# time=1100, select=0 shift_value=011 din=00000001 dout=00001000
# time=1200, select=1 shift_value=000 din=00000000 dout=00000000
# time=1300, select=1 shift_value=001 din=10111001 dout=01110011
# time=1400, select=1 shift_value=010 din=10111001 dout=11100110
# time=1500, select=1 shift_value=011 din=10111001 dout=11001101
# ** Note: $stop : testbench.sv(79)
# Time: 1600 ns Iteration: 0 Instance: /barrel_shifter_testbench
# Break in Module barrel_shifter_testbench at testbench.sv line 79
# Stopped at testbench.sv line 79
# exit
# End time: 18:47:47 on Feb 07,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
```

I went with the 3-stage mux per bit design, but I left the change direction to some if statements that checked the direction instead of storing a separate string of bits based on inversion or not- it made more sense for me to be able to see what was happening when going left. I did look at an inversion method, and it makes sense not have to run a check on direction at each operation, but I don't need space for an extra variable the way I did it. As you can see above, the print statements given in the first testbench do a great job of showing that the output works as desired with the 1 rolling right and left for logical and the ones coming around for rotation. I also followed the specs and made it work with 8-bit but did not hard code 8 into the parameter, although I didn't implement a nested loop so it would actually work with larger numbers because I have a set number of stages, but I do know that all I would have to do is convert from the 3 separate stages to looping over the stages. Below is the post-mapping schematic, where you can clearly see the three separate stages needed for the 8-bit barrel shifter.

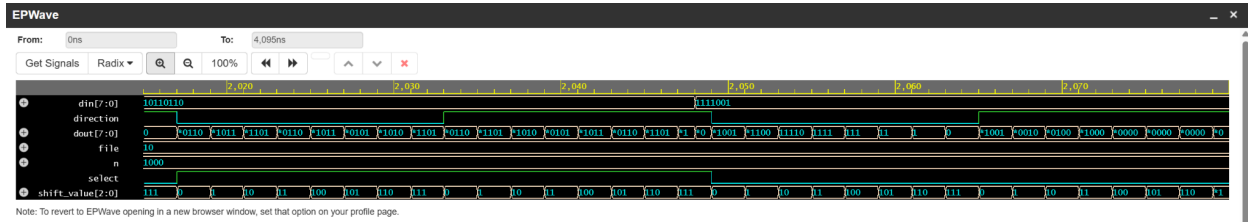


(Purely behavioral code, mux implementation in the code submission):

```
module barrel_shifter #(
    parameter int n = 8
)(
    input logic [$clog2(n)-1:0] shift_value,
    input logic direction,
    input logic [n-1:0] din,
    input logic select,
    output logic [n-1:0] dout
);

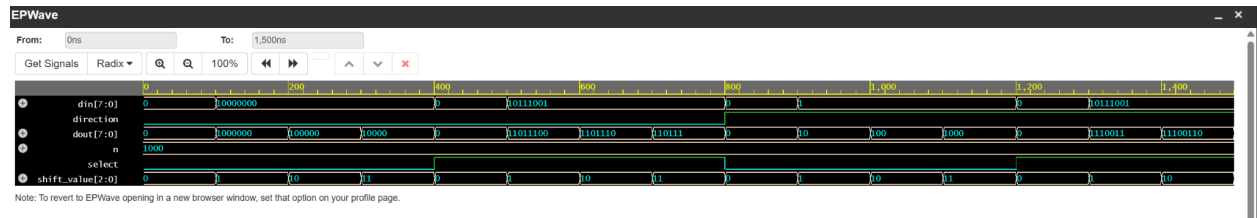
always_comb begin
    if (direction) begin
        dout = select ? (din << shift_value) | (din >> (n - shift_value)) : (din << shift_value);
    end else begin
        dout = select ? (din >> shift_value) | (din << (n - shift_value)) : (din >> shift_value);
    end
end

endmodule
```



```
# time=4061, select=0 shift_value=110 din=01111001 dout=01000000
# select = 0, direction = 1, shift_value = 6, din=01111001, dout=01000000
# time=4063, select=0 shift_value=111 din=01111001 dout=10000000
# select = 0, direction = 1, shift_value = 7, din=01111001, dout=10000000
# time=4065, select=1 shift_value=000 din=01111001 dout=01111001
# select = 1, direction = 0, shift_value = 0, din=01111001, dout=01111001
# time=4067, select=1 shift_value=001 din=01111001 dout=10111100
# select = 1, direction = 0, shift_value = 1, din=01111001, dout=10111100
# time=4069, select=1 shift_value=010 din=01111001 dout=01011110
# select = 1, direction = 0, shift_value = 2, din=01111001, dout=01011110
# time=4071, select=1 shift_value=011 din=01111001 dout=00101111
# select = 1, direction = 0, shift_value = 3, din=01111001, dout=00101111
# time=4073, select=1 shift_value=100 din=01111001 dout=10010111
# select = 1, direction = 0, shift_value = 4, din=01111001, dout=10010111
# time=4075, select=1 shift_value=101 din=01111001 dout=11001011
# select = 1, direction = 0, shift_value = 5, din=01111001, dout=11001011
# time=4077, select=1 shift_value=110 din=01111001 dout=11100101
# select = 1, direction = 0, shift_value = 6, din=01111001, dout=11100101
# time=4079, select=1 shift_value=111 din=01111001 dout=11110010
# select = 1, direction = 0, shift_value = 7, din=01111001, dout=11110010
# time=4081, select=1 shift_value=000 din=01111001 dout=01111001
# select = 1, direction = 1, shift_value = 0, din=01111001, dout=01111001
# time=4083, select=1 shift_value=001 din=01111001 dout=11110010
# select = 1, direction = 1, shift_value = 1, din=01111001, dout=11110010
# time=4085, select=1 shift_value=010 din=01111001 dout=11100101
# select = 1, direction = 1, shift_value = 2, din=01111001, dout=11100101
# time=4087, select=1 shift_value=011 din=01111001 dout=11001011
# select = 1, direction = 1, shift_value = 3, din=01111001, dout=11001011
# time=4089, select=1 shift_value=100 din=01111001 dout=10010111
# select = 1, direction = 1, shift_value = 4, din=01111001, dout=10010111
# time=4091, select=1 shift_value=101 din=01111001 dout=00101111
# select = 1, direction = 1, shift_value = 5, din=01111001, dout=00101111
# time=4093, select=1 shift_value=110 din=01111001 dout=01011110
# select = 1, direction = 1, shift_value = 6, din=01111001, dout=01011110
# time=4095, select=1 shift_value=111 din=01111001 dout=10111100
# select = 1, direction = 1, shift_value = 7, din=01111001, dout=10111100
# ** Note: $finish : testbench.sv(35)
# Time: 4196 ns Iteration: 0 Instances: /barrel_shifter_testbench2
```

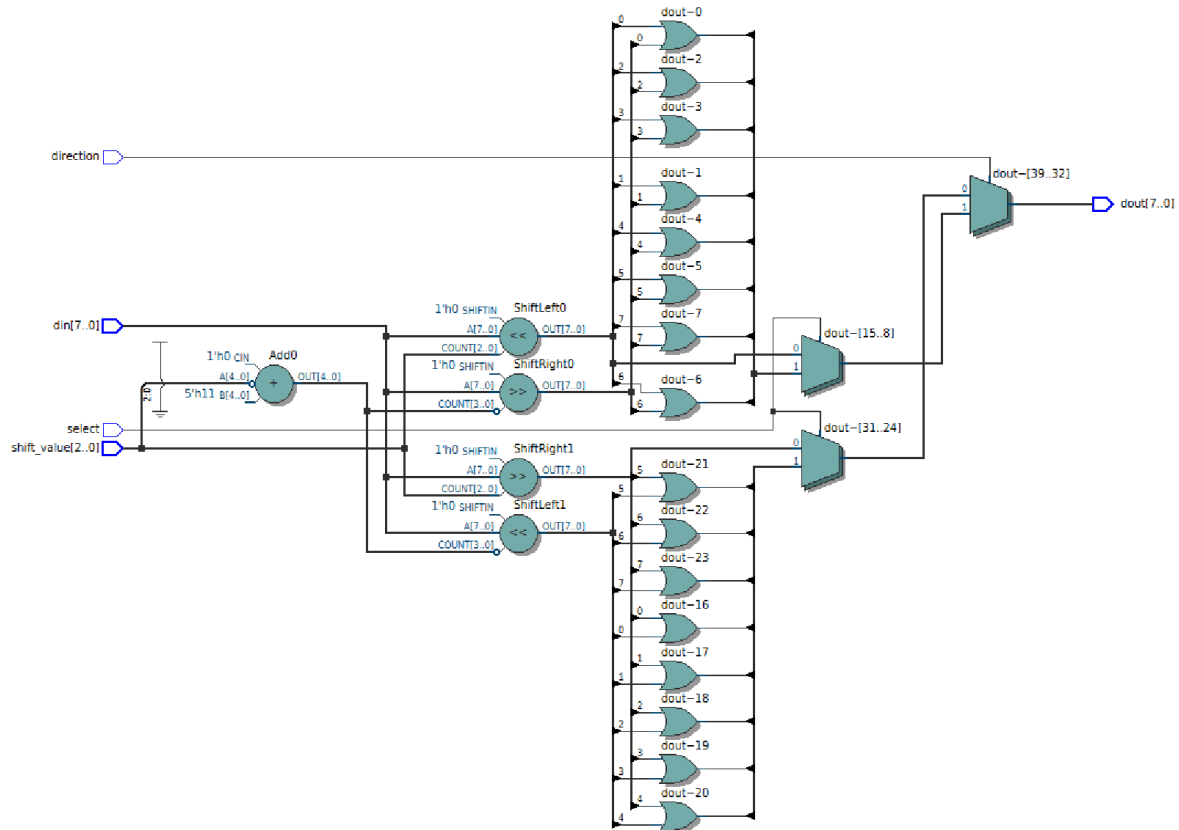
You can look through the above to see that it worked but it's a bit harder to see, so below I have the first one where the signals are easier to read.



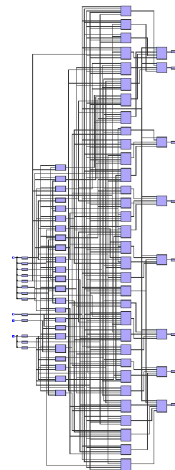
```
# run -all
# time=0,  select=0  shift_value=000  din=00000000  dout=00000000
# time=100, select=0  shift_value=001  din=10000000  dout=01000000
# time=200, select=0  shift_value=010  din=10000000  dout=00100000
# time=300, select=0  shift_value=011  din=10000000  dout=00010000
# time=400, select=1  shift_value=000  din=00000000  dout=00000000
# time=500, select=1  shift_value=001  din=10111001  dout=11011100
# time=600, select=1  shift_value=010  din=10111001  dout=01101110
# time=700, select=1  shift_value=011  din=10111001  dout=00110111
# time=800, select=0  shift_value=000  din=00000000  dout=00000000
# time=900, select=0  shift_value=001  din=00000001  dout=00000010
# time=1000, select=0  shift_value=010  din=00000001  dout=00000100
# time=1100, select=0  shift_value=011  din=00000001  dout=00001000
# time=1200, select=1  shift_value=000  din=00000000  dout=00000000
# time=1300, select=1  shift_value=001  din=10111001  dout=01110011
# time=1400, select=1  shift_value=010  din=10111001  dout=11100110
# time=1500, select=1  shift_value=011  din=10111001  dout=11001101
```

As you can see, it works just the same as the mux code.

As for the netlist design:



The design looks much simpler because there are shifter modules, so you just perform a shift operation based on the condition, whereas in the other method, we built the shift operation with multiplexors.



The post mapping, on the other hand, is quite interesting...